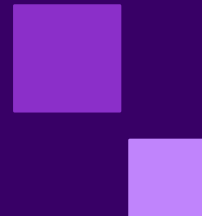


COURSE 03 — DART FUNDAMENTALS

Dart Fundamentals

The language powering Flutter — mastered from zero

Instructor: Islam Amour Mohamed | Phase 03 of 12



40 Slides

11 Modules

What You Will Learn

01

Introduction

Why Dart, JIT vs AOT, Hot Reload

03

var vs dynamic vs Object

Type safety & flexibility

05

Null Safety

?, !, ??, late — all operators covered

07

Collections

List, Set, Map, if/for in collections

09

Async Programming

Future, async/await, error handling

02

Variables & Types

Primitives, var, String interpolation

04

final vs const

Runtime vs compile-time constants

06

Functions

Arrow, named params, closures

08

Exception Handling

try / catch / finally / throw

10

Streams

Stream vs Future, periodic, Flutter use

11

Challenge

Simple VM, Dart, and Flutter project

MODULE

01

Author: Mezenner Fares

Introduction to Dart

Why Dart powers the Flutter ecosystem

- Why Dart?
- Why Flutter Chose Dart
- JIT vs AOT
- Hot Reload
- Similarities with Other Languages

Why Dart?

Dart is a strongly-typed, object-oriented language developed by Google. Designed to be easy to learn for developers coming from Java or C#, while being more modern and productive. Dart compiles to native machine code for mobile/desktop and to JavaScript for the web.



Strongly Typed

Type errors caught at compile time, not runtime



Object-Oriented

Classes, interfaces, generics — familiar to Java/C# devs



Multi-Target

Compiles to native ARM (mobile) and JavaScript (web)



Modern & Productive

Clean syntax, null safety, async/await built-in

Why Flutter Chose Dart

Flutter chose Dart because it supports both JIT and AOT compilation — no other language offered both when Flutter was designed.

JIT — Development Mode

Just-In-Time compilation

- Code compiled at runtime
- Enables Hot Reload instantly
- Fast development feedback loop
- Used in debug/development builds

AOT — Production Mode

Ahead-Of-Time compilation

- Code compiled before execution
- Produces fast native binaries
- Optimized for release builds
- Used in production deployments

JIT vs AOT — Side by Side

	JIT	AOT
When	At runtime	Before execution
Speed	Slower startup	Faster startup
Output	Interpreted bytecode	Native machine code
Use Case	Development, Hot Reload	Production release
Debugging	Full debug info available	Limited debug info
Dart Mode	dart run / flutter run	flutter build / release

Dart uses JIT in development mode and AOT in production mode — the best of both worlds.

Hot Reload

Hot Reload injects updated source code into the running Dart VM without restarting the app. UI changes appear in under a second, dramatically speeding up development.

How Hot Reload Works



Key Benefits:

- Preserves the app state
- Sub-second feedback loop
- Possible only with JIT compilation

Dart vs Other Languages

Feature	Dart	Java	Python
Typing	Static	Static	Dynamic
Null Safety	Sound (built-in)	Optional	None
Compilation	JIT + AOT	JIT (JVM)	Interpreted
Syntax	Modern, clean	Verbose	Minimal
OOP	Full support	Full support	Multi-paradigm
Async	async/await	CompletableFuture	asyncio
Flutter	Native ✓	Bridge layer	Not supported

Dart feels familiar to Java/C# developers while offering modern features like null safety and first-class async support.

MODULE

02

Variables & Types

The building blocks of every Dart program

- Primitive Types: int, double, num, bool, String
- Type Inference with var
- String Interpolation

Primitive Types in Dart

All types in Dart are objects — even `int` and `bool`.

int

```
int age = 25;
```

Whole numbers. No decimals.

double

```
double price = 9.99;
```

Decimal numbers (64-bit float).

num

```
num score = 88;
```

Parent of `int` and `double`.

bool

```
bool isLoggedIn = true;
```

Only true or false.

String

```
String name = 'Ali';
```

A sequence of UTF-16 characters.

Type Inference & String Interpolation

Type Inference with var

Dart infers the type from the assigned value. Once inferred, the type is fixed at compile time.

Example

```
var name = 'Ali';    // String
var age = 25;        // int

// ERROR: type mismatch!
// name = 42; // can't assign int to String
```



var is safe

Type is locked at compile time. Errors appear before running.

String Interpolation

Embed variables directly in strings using \$. For expressions, use \${}. More readable than concatenation.

Example

```
String name = 'Ali';
int age = 25;

print('Hello $name');
// Hello Ali
print('Next year: ${age + 1}');
// Next year: 26
```

Use \${} for expressions

\$variable for simple access, \${expr} for any computation.

MODULE

03

var vs dynamic vs Object

Understanding Dart's type system flexibility

- var — compile-time type inference
- dynamic — disables type checking
- Object — type-safe flexibility
- Comparison summary

var vs dynamic vs Object

var

Compile-Time Inference

Type is inferred once and fixed. Type errors caught at compile time.

```
var name = 'Ali'; // String
// name = 42; // ERROR at compile
time
```

✓ Safe for local variables

dynamic

Disables Type Checking

Can hold any type. Reassignable to different types. Errors only at runtime.

```
dynamic value = 'Hello';
value = 42; // OK
value = true; // OK
// Runtime errors possible!
```

⚠ Use sparingly

Object

Type-Safe Flexibility

Can hold any type but type checking is enforced. Cast required to use methods.

```
Object value = 'Hello';
// value.toUpperCase(); // ERROR
if (value is String) {
  print(value.toUpperCase()); //
  ✓
}
```

✓ Safe, requires cast

MODULE

04

final vs const

When and why to lock values in Dart

- final — runtime constant (assigned once)
- const — compile-time constant (deep immutable)
- Why Flutter uses const widgets

final vs const

final

Runtime Constant

Assigned only once. Value can be determined at runtime — from a function call, user input, or any runtime expression.

Example

```
final now = DateTime.now();  
final name = 'Ali';  
  
// ERROR: can't reassign  
// now = DateTime.now();
```

Value set at runtime. Can't be reassigned. Objects can still be mutable.

const

Compile-Time Constant

Value must be known at compile time. Deeply immutable — the entire object is frozen. Identical const objects share memory.

Example

```
const pi = 3.14159;  
const appName = 'MyApp';  
  
// ERROR: not compile-time!  
// const now = DateTime.now();
```

Value known at compile time. Deeply frozen. Same const value = same memory object.

Why Flutter Uses const Widgets

Marking a widget as const tells Flutter: "this widget will never change." Flutter can skip rebuilding it on every render cycle, improving performance.

Without const

Widget may be rebuilt on every setState() even if nothing changed.

```
// Rebuilt unnecessarily
Text('Hello World')

Column(
  children: [
    Text('Static text'), // rebuilt
  ],
)
```

With const

Flutter detects the widget is identical and skips the rebuild entirely.

```
// Skipped on rebuild ✓
const Text('Hello World')

Column(
  children: [
    const Text('Static text'), // ✓
  ],
)
```

As a rule: mark every widget const if it has no dynamic data. Use it everywhere in Flutter code.

MODULE

05

Null Safety

Eliminating null errors at compile time

- Why Null Safety?
- Non-Nullable vs Nullable Types (?)
- Null Assertion (!)
- Null-Aware Operators (??, ??=, ?.)
- late Keyword

Why Null Safety?

The Problem: Before Null Safety, any variable could be null — causing `NullPointerException` crashes at runtime. Dart's Sound Null Safety moves null errors from runtime to compile time.

Before Null Safety (Java)

```
// Java - crashes at RUNTIME
String name = null;
name.length();
// NullPointerException! ✖

// Found only when running
// Cannot predict at compile time
```

⚠ Error found at runtime

With Null Safety (Dart)

```
// Dart - caught at COMPILE TIME
String name = 'Ali';
// name = null; // ERROR ✓

String? nullable = null; // OK
// Use ? to allow null explicitly
```

✅ Error found at compile time

Null Safety Operators

?**Nullable Type**

Allows null

```
String? name; // null OK
```

??**Null-Aware Fallback**

Provide default if null

```
name ?? 'Guest' // use Guest if null
```

?.**Safe Access**

Call on nullable — returns null safely

```
name?.length // null if name is null
```

!**Null Assertion**

Assert non-null (runtime risk)

```
print(name!.length); // trust me
```

??=**Null-Aware Assign**

Assign only if currently null

```
name ??= 'Ali'; // assign if null only
```

late**Late Init**

Promise to init before use

```
late String user;  
user = 'Ali'; // set later
```

MODULE

06

Functions

Dart's first-class functions & closures

- Normal functions with return types
- Arrow syntax ($\Rightarrow$)
- Named parameters & defaults
- Optional positional parameters ($[]$)
- Closures & anonymous functions

Normal Functions & Arrow Syntax

Normal Function

Return type, name, parameters, body. Functions are first-class objects in Dart.

Dart

```
int add(int a, int b) {  
  return a + b;  
}  
  
void greet(String name) {  
  print('Hello, $name');  
}  
  
print(add(3, 4)); // 7  
greet('Ali');    // Hello, Ali
```

Arrow Syntax (=>)

Shorthand for single-expression functions. Very common in Flutter callbacks and widget builders.

Dart

```
// Arrow = shorthand for single expression  
int add(int a, int b) => a + b;  
  
String greet(String name)  
  => 'Hello, $name!';  
  
print(add(2, 3));    // 5  
print(greet('Ali')); // Hello, Ali!
```

Named & Optional Parameters

Named Parameters { }

Wrapped in {}. Optional by default. Use required to make mandatory. Flutter widget constructors use named params exclusively.

Dart

```
void createUser({  
  required String name,  
  int age = 18,  
}) {  
  print('$name is $age years old');  
}  
  
createUser(name: 'Ali');  
// Ali is 18 years old  
createUser(name: 'Sara', age: 25);  
// Sara is 25 years old
```

Optional Positional []

Wrapped in []. Must come after required params. Positional — order matters. Provide default values.

Dart

```
void greet(String name, [String title = 'Mr']) {  
  print('Hello, $title $name');  
}  
  
greet('Ali');  
// Hello, Mr Ali  
greet('Sara', 'Dr');  
// Hello, Dr Sara
```

Closures & Anonymous Functions

A closure is an anonymous function that captures variables from its surrounding scope. Closures are first-class objects in Dart — used heavily in collection methods and Flutter callbacks.

Dart

```
// Closure assigned to variable
var double = (int x) => x * 2;
print(double(5)); // 10

// Closure captures outer variable
int counter = 0;
var increment = () => counter++;
increment();
increment();
print(counter); // 2

// Common: forEach callback
List<int> nums = [1, 2, 3];
nums.forEach((n) => print(n * 2));
// 2, 4, 6
```

Where Closures Appear

Event Callbacks

```
onPressed: () => doSomething()
```

Collection Methods

```
list.map((x) => x * 2)
```

Async Handlers

```
future.then((v) => print(v))
```

Builder Functions

```
builder: (ctx) => MyWidget()
```

MODULE

07

Collections

List, Set, Map — and Dart's powerful collection syntax

- List — ordered, indexed collection
- Set — unique items, no duplicates
- Map — key-value pairs (dict)
- Collection if — conditional items
- Collection for — generated items
- Spread operators ... and ...?

List & Set

List<T>

Ordered. Indexed. Allows duplicates.

Dart

```
List<String> fruits =  
  ['Apple', 'Banana', 'Cherry'];  
  
fruits.add('Date');  
fruits.remove('Banana');  
print(fruits.first); // Apple  
print(fruits.last);  // Date  
  
// Filter  
var long = fruits  
  .where((f) => f.length > 4)  
  .toList(); // ['Apple', 'Cherry']  
  
// Transform  
var upper = fruits  
  .map((f) => f.toUpperCase())  
  .toList();
```

Set<T>

Unordered. Unique items only.

Dart

```
Set<String> tags =  
  {'flutter', 'dart', 'mobile'};  
  
// Duplicate ignored  
tags.add('flutter');  
print(tags);  
// {flutter, dart, mobile}  
  
// Dedup a list  
List<int> nums = [1, 2, 2, 3, 3];  
Set<int> unique = nums.toSet();  
print(unique); // {1, 2, 3}  
  
// Convert back to list  
unique.toList();
```

Map — Key-Value Pairs

A Map stores key-value pairs. Keys must be unique. Equivalent to a Python dictionary or Java HashMap. Used heavily in Dart for representing structured data like JSON.

Map Operations

```
Map<String, int> scores = {'Ali': 90, 'Sara': 85};

// Create / add
scores['Omar'] = 78;

// Read
print(scores['Ali']); // 90

// Update
scores['Sara'] = 95;

// Delete
scores.remove('Omar');

// Iterate
scores.forEach((name, score) {
  print('$name: $score');
});
```

Common Methods

<code>.keys</code>	All keys as Iterable
<code>.values</code>	All values as Iterable
<code>.containsKey()</code>	Check if key exists
<code>.putIfAbsent()</code>	Add only if key missing
<code>.entries</code>	Iterate key-value pairs

Collection if, for & Spread

Collection if

Conditionally include items in a list literal

```
bool isAdmin = true;

List<String> menu = [
    'Home',
    'Profile',
    if (isAdmin)
        'Admin Panel',
];
// [Home, Profile,
//  Admin Panel]
```

Collection for

Generate list items using a for loop inline

```
List<int> nums = [1, 2, 3, 4];

List<int> doubled = [
    for (var n in nums)
        n * 2
];
// [2, 4, 6, 8]

List<String> labels = [
    for (var n in nums)
        'Item $n'
];
```

Spread ...

Insert all elements of a collection into another

```
List<String> cats =
    ['Luna', 'Mochi'];
List<String> dogs =
    ['Max', 'Rex'];

List<String> pets =
    [...cats, ...dogs];
// [Luna, Mochi, Max, Rex]

// Null-aware spread
List<String>? extras = null;
List<String> all =
    [...pets, ...?extras];
```

MODULE

08

Exception Handling

Managing runtime errors gracefully

- try — wrap risky code
- catch — handle the error
- finally — always runs
- throw — raise exceptions
- on — catch specific exception types

try / catch / finally / throw

try

Wraps risky code that may throw

catch

Handles the error if one occurs

finally

Always runs — success or failure

throw

Manually raise an exception

Full Example

```
void checkAge(int age) {
    if (age < 0)
        throw ArgumentError('Age cannot be negative');
}

try {
    checkAge(-1);
} catch (e) {
    print('Error: $e');
    // Error: Invalid argument: Age cannot be negative
} finally {
    print('Done'); // Always runs
}
```

Specific Types

```
// Catch specific types
try {
    int result =
        int.parse('abc');
} on FormatException catch (e) {
    print('Bad format: $e');
} on RangeError catch (e) {
    print('Range error: $e');
} catch (e) {
    print('Other: $e');
}
```

MODULE

09

Async Programming

Non-blocking code with Future & async/await

- Synchronous vs Asynchronous
- Future — single async value
- async / await — readable async code
- Handling async errors with try/catch

Future & async / await

Dart is single-threaded but non-blocking. It uses an event loop to handle async operations. A Future represents a single async value available later.

Future<T>

```
Future<String> fetchUsername() {  
  return Future.delayed(  
    Duration(seconds: 1),  
    () => 'Ali',  
  );  
}  
  
// With .then()  
fetchUsername()  
  .then((name) => print(name));  
// Ali (after 1 second)
```

async / await

```
Future<String> fetchUsername() async {  
  await Future.delayed(  
    Duration(seconds: 1));  
  return 'Ali';  
}  
  
Future<void> main() async {  
  print('Fetching...');  
  String name = await fetchUsername();  
  print('Hello, $name'); // Hello, Ali  
}
```

Handling Async Errors

Errors in async functions are caught with try/catch just like synchronous code — as long as you use await. Without await, errors go unhandled.

With await — Correct

```
Future<void> loadData() async {  
  try {  
    String result = await fetchUsername();  
    print(result);  
  } catch (e) {  
    print('Failed: $e');  
  } finally {  
    print('Cleanup done');  
  }  
}
```

Key Rules

- ✓ Always use await in try/catch
- ✓ Mark async functions with async
- ⚠ .then() errors use .catchError()
- ✓ finally works the same as sync

MODULE

10

Streams

Sequences of asynchronous events over time

- Future vs Stream — one vs many values
- `Stream.periodic` — timed events
- `async*` and `yield` — stream generators
- Where Streams appear in Flutter (BLoC, Firebase)

Future vs Stream



Future

One delivery
(single value)



Stream

Live feed
(multiple values)

Future<T>

```
// One result, one time
Future<int> getScore() async
=> 95;

// Use with await
int score = await getScore();
print(score); // 95
```

Stream<T>

```
// Multiple results over time
Stream<int> countUp() async* {
  for (int i = 1; i <= 5; i++) {
    await Future.delayed(
      Duration(seconds: 1));
    yield i;
  }
}
```

Streams in Flutter

Firestore

Listen to database changes live without polling. The stream emits a new value every time data changes.

```
db.collection('users')
  .snapshots()
  .listen((snapshot) {
    print(snapshot.docs.length);
  });
```

BLoC State Management

The BLoC pattern uses streams to emit state changes. UI subscribes and rebuilds when new state arrives.

```
myBloc.state.listen((state) {
  if (state is LoadedState) {
    // rebuild UI
  }
});
```

WebSockets

Real-time bidirectional communication. Each message from the server is a new stream event.

```
channel.stream.listen((message) {
  print('Received: $message');
},
  onError: (e) => print(e),
  onDone: () => print('Closed'),
);
```

MODULE

11

Challenge Project

Put it all together in a real Dart project

- Student Management System
- Classes, null safety, named params
- Collections with if/for
- Async loading & error handling
- Null-aware operators in practice

Student Management System

Build a console app using Maps & Lists — no classes. Students are stored as `Map<String, dynamic>`. The app filters, computes averages, and loads async — with null safety and error handling throughout.



Maps, Not Classes

Each student is a `Map<String, dynamic>` with 'name', 'grade', 'email'



Async Data Loading

`loadStudents()` simulates a network call with `Future.delayed`



Filter & Collection for

Top students with `.where()`, names with collection for



Average with a loop

for loop + accumulator, throw if list is empty



Error Handling

`try/catch/finally` wraps the entire async operation



Null-Aware Operators

`email ?? 'no email' — String? cast from dynamic map value`

Challenge: Student System — Part 1

Data Structure

```
List<Map<String, dynamic>> students = [
  {
    'name': 'Ali',
    'grade': 90.0,
    'email': 'ali@mail.com'
  },
  {
    'name': 'Sara',
    'grade': 85.0,
    'email': null      // nullable
  },
  {
    'name': 'Omar',
    'grade': 78.0,
    'email': 'omar@mail.com'
  },
];
```

Async Loader & Average

```
// Simulate async network load
Future<List<Map<String, dynamic>>>
  loadStudents() async {
    await Future.delayed(
      Duration(seconds: 1));
    return students;
  }

// Compute average with for loop
double computeAverage(
  List<Map<String, dynamic>> list) {
  if (list.isEmpty)
    throw Exception('No students found');
  double total = 0;
  for (var s in list) {
    total += s['grade'] as double;
  }
  return total / list.length;
}
```

Challenge: Student System — Part 2

main() Function

```
Future<void> main() async {
  try {
    print('Loading...');
    var data = await loadStudents();

    // Collection for – extract names
    List<String> names = [
      for (var s in data) s['name'] as String
    ];
    print('Students: $names');

    // Filter top students (grade >= 85)
    var top = data
      .where((s) => (s['grade'] as double) >= 85)
      .toList();
    print('Top: ${[for (var s in top) s['name']]}');

    // Average with helper function
    double avg = computeAverage(data);
    print('Class average: $avg');

    // Null-aware email print
    for (var s in data) {
      String? email = s['email'] as String?;
      print('${s['name']}: ${email ?? 'no email'}');
    }
  }
}
```

Concepts Used

- ✓ Maps as data structures
- ✓ Dynamic casting (as Type)
- ✓ Null safety (String?)
- ✓ Async / await
- ✓ Collection for / .where()
- ✓ for loop + accumulator
- ✓ try / catch / finally
- ✓ Null-aware ?? operator
- ✓ String interpolation \${}

You've Mastered Dart!

Every concept you learned here directly powers Flutter widget development.

Phase 04

**OOP
with Dart**

5h

Phase 05

**First Flutter
App**

4h

Phase 06

**Flutter Under
the Hood**

3h

Phase 07

**Building
Static UIs**

7h

Flutter Developer Training Program

Instructor: Islam Amour Mohamed · 55 Hours · 12 Phases · 1 Capstone Project

"Dart is not just a language — it's the foundation everything in Flutter is built on."